Thực hành tích hợp AJAX vào module Room

Sinh viên: Lê Xuân Toàn - 23A1001D0361 - Nhóm 7.

1 Phạm vi

Module FS01 và FS02: quản lý phòng họp và cơ sở vật chất. Entity chính là Room. Bổ sung AJAX vào dự án ASP.NET Core MVC hiện có; không tạo DbContext hoặc migration mới vì cấu trúc dữ liệu không đổi.

Trang chính: /Admin/Room. Bản truyền thống giữ tại /Admin/Room/Traditional để so sánh.

2 Đối chiếu yêu cầu

UI Update bằng XMLHttpRequest: load() trong wwwroot/js/room-ajax.js tạo XMLHttpRequest GET ListJson. render() nhận mảng items và dựng tbody bằng createElement/textContent, không ghép input vào innerHTML.

Create và Delete bằng jQuery AJAX: write() dùng $.ajax POST SaveJson và DisableJson. Form serialize kèm antiforgery token. Delete là xóa mềm IsDisabled = true, giữ lịch sử cuộc họp. Khôi phục gọi RestoreJson; sửa cũng dùng SaveJson.

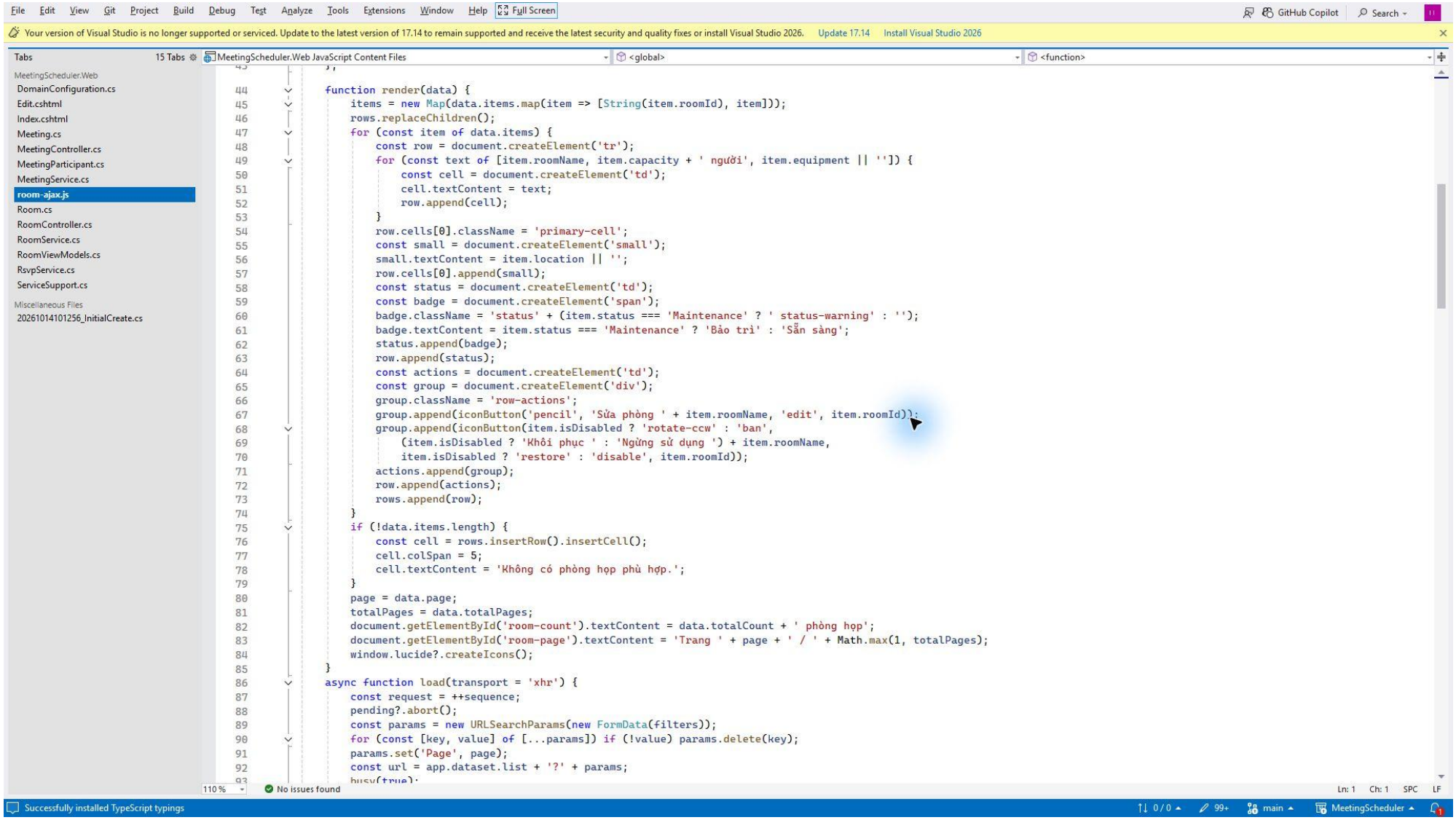
Pagination EF Core: RoomService.QueryAsync -> ServiceSupport.PageAsync -> CountAsync -> Skip((page - 1) * size) -> Take(size) -> ToListAsync. Nút trước/sau gọi XMLHttpRequest; JSON trả totalCount/page/pageSize/totalPages. Khi trang vượt phạm vi, controller tải lại trang cuối hợp lệ.

AJAX Search bằng fetch(): tìm kiếm debounce 300 ms hoặc bấm Lọc -> fetch(ListJson?SearchString=...) -> response.json() -> render(). Tìm kiếm, thay bộ lọc hoặc số dòng đưa Page về 1; phân trang giữ toàn bộ bộ lọc hiện tại.

Phân tích luồng: UI -> JavaScript -> RoomController -> RoomService -> EF Core -> Database -> JSON -> JavaScript -> DOM.

Ảnh code 1 Render bảng từ JSON

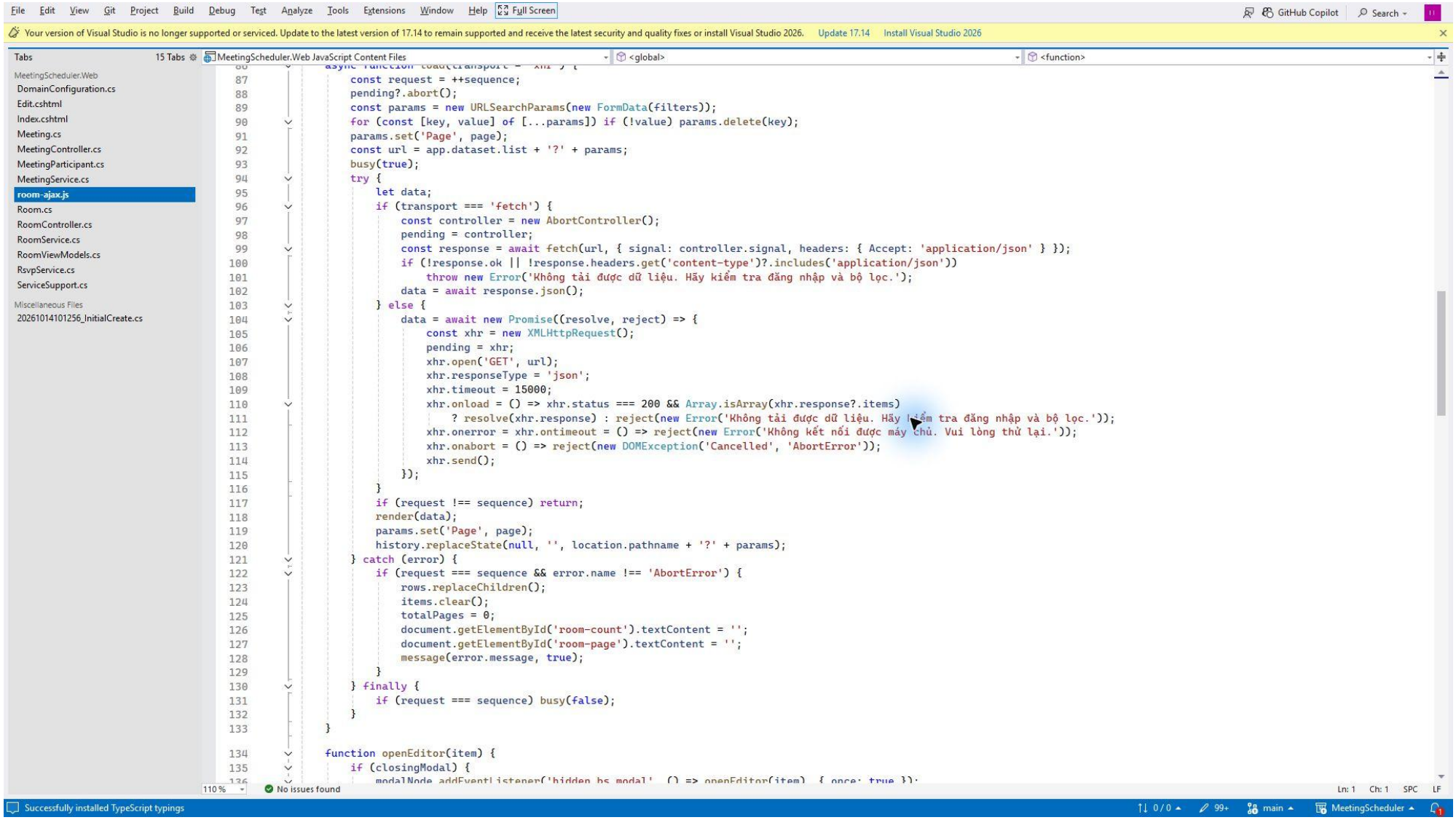
File: wwwroot/js/room-ajax.js. Yêu cầu UI Update: render(data) dùng createElement và textContent để dựng bảng, trạng thái và số trang; dữ liệu không được ghép vào innerHTML.



```
function render(data) {
    items = new Map(data.items.map(item => [String(item.roomId), item]));
    rows.replaceChildren();
    for (const item of data.items) {
        const row = document.createElement('tr');
        for (const text of [item.roomName, item.capacity + ' người', item.equipment || '']) {
            const cell = document.createElement('td');
            cell.textContent = text;
            row.append(cell);
        }
        row.cells[0].className = 'primary-cell';
        const small = document.createElement('small');
        small.textContent = item.location || '';
        row.cells[0].append(small);
        const status = document.createElement('td');
        const badge = document.createElement('span');
        badge.className = 'status' + (item.status === 'Maintenance' ? ' status-warning' : '');
        badge.textContent = item.status === 'Maintenance' ? 'Bảo trì' : 'Sẵn sàng';
        status.append(badge);
        row.append(status);
        const actions = document.createElement('td');
        const group = document.createElement('div');
        group.className = 'row-actions';
        group.append(iconButton('pencil', 'Sửa phòng ' + item.roomName, 'edit', item.roomId));
        group.append(iconButton(item.isDisabled ? 'rotate-ccw' : 'ban',
            (item.isDisabled ? 'Khôi phục ' : 'Ngừng sử dụng ') + item.roomName,
            item.isDisabled ? 'restore' : 'disable', item.roomId));
        actions.append(group);
        row.append(actions);
        rows.append(row);
    }
    if (!data.items.length) {
        const cell = rows.insertRow().insertCell();
        cell.colSpan = 5;
        cell.textContent = 'Không có phòng họp phù hợp.';
    }
    page = data.page;
    totalPages = data.totalPages;
    document.getElementById('room-count').textContent = data.totalCount + ' phòng họp';
    document.getElementById('room-page').textContent = 'Trang ' + page + ' / ' + Math.max(1, totalPages);
    window.lucide?.createIcons();
}
async function load(transport = 'xhr') {
    const request = ++sequence;
    pending?.abort();
    const params = new URLSearchParams(new FormData(filters));
    for (const [key, value] of [...params]) if (!value) params.delete(key);
    params.set('Page', page);
    const url = app.dataset.list + '?' + params;
```

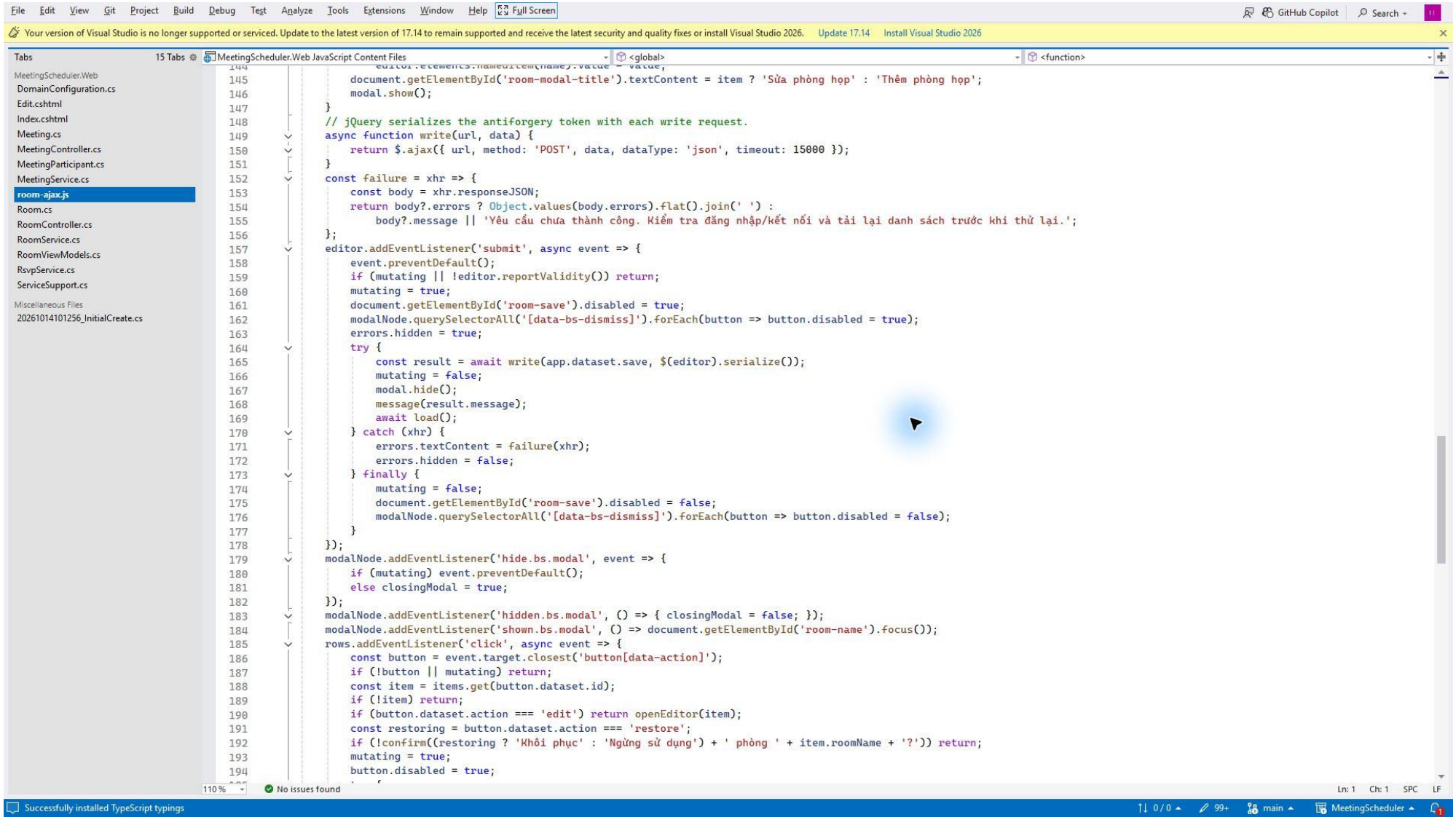
Ảnh code 2 XMLHttpRequest và fetch

File: wwwroot/js/room-ajax.js. Lần đầu và phân trang dùng XMLHttpRequest. Tìm kiếm dùng fetch, response.json và gọi render(data). sequence và abort ngăn phản hồi cũ ghi đè kết quả mới.



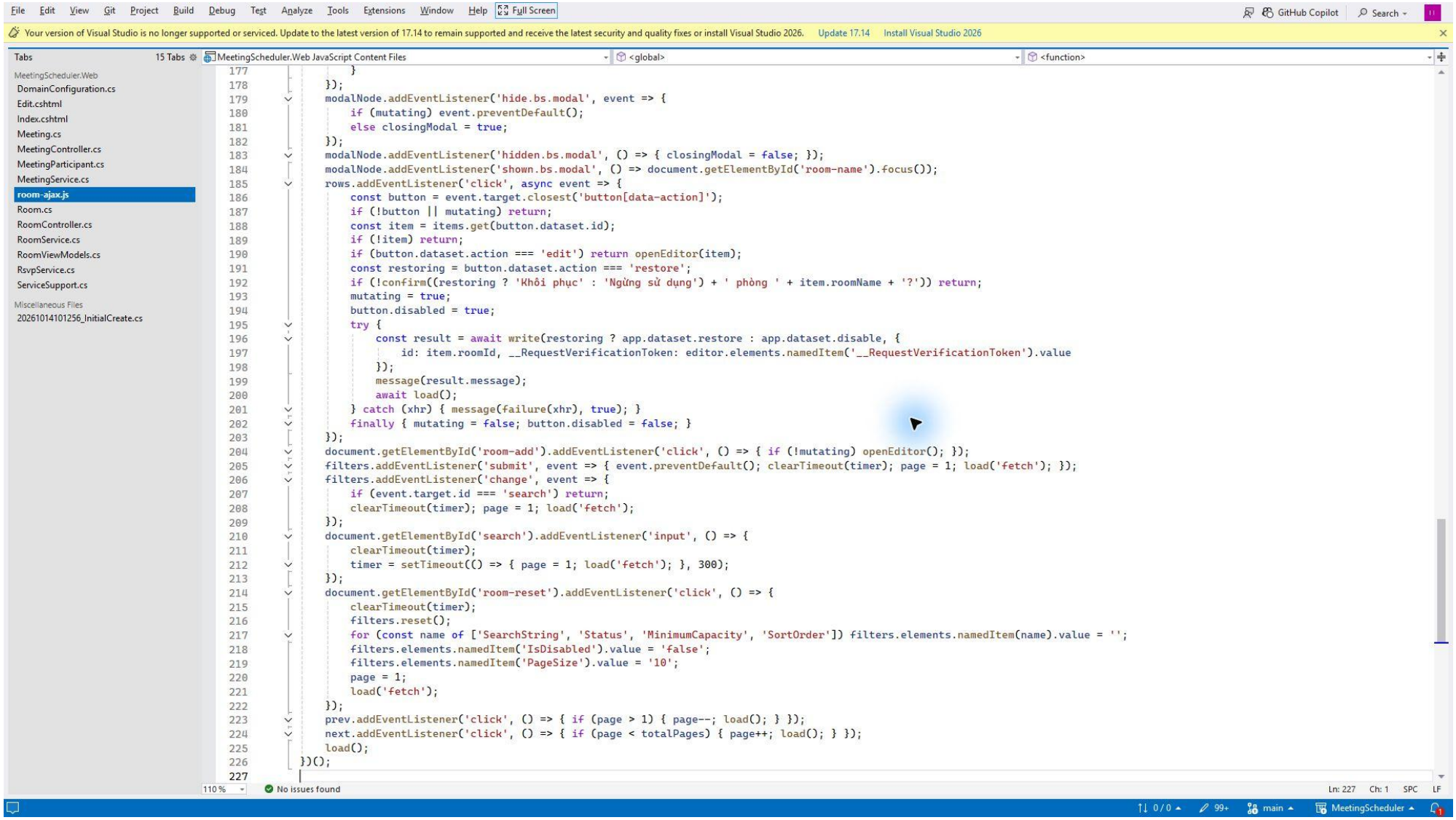
Ảnh code 3 Thêm và sửa bằng jQuery AJAX

File: wwwroot/js/room-ajax.js. write() dùng $.ajax POST. Submit bị chặn điều hướng bằng preventDefault; serialize gửi biểu mẫu và token; JSON thành công đóng modal và tải lại bảng.



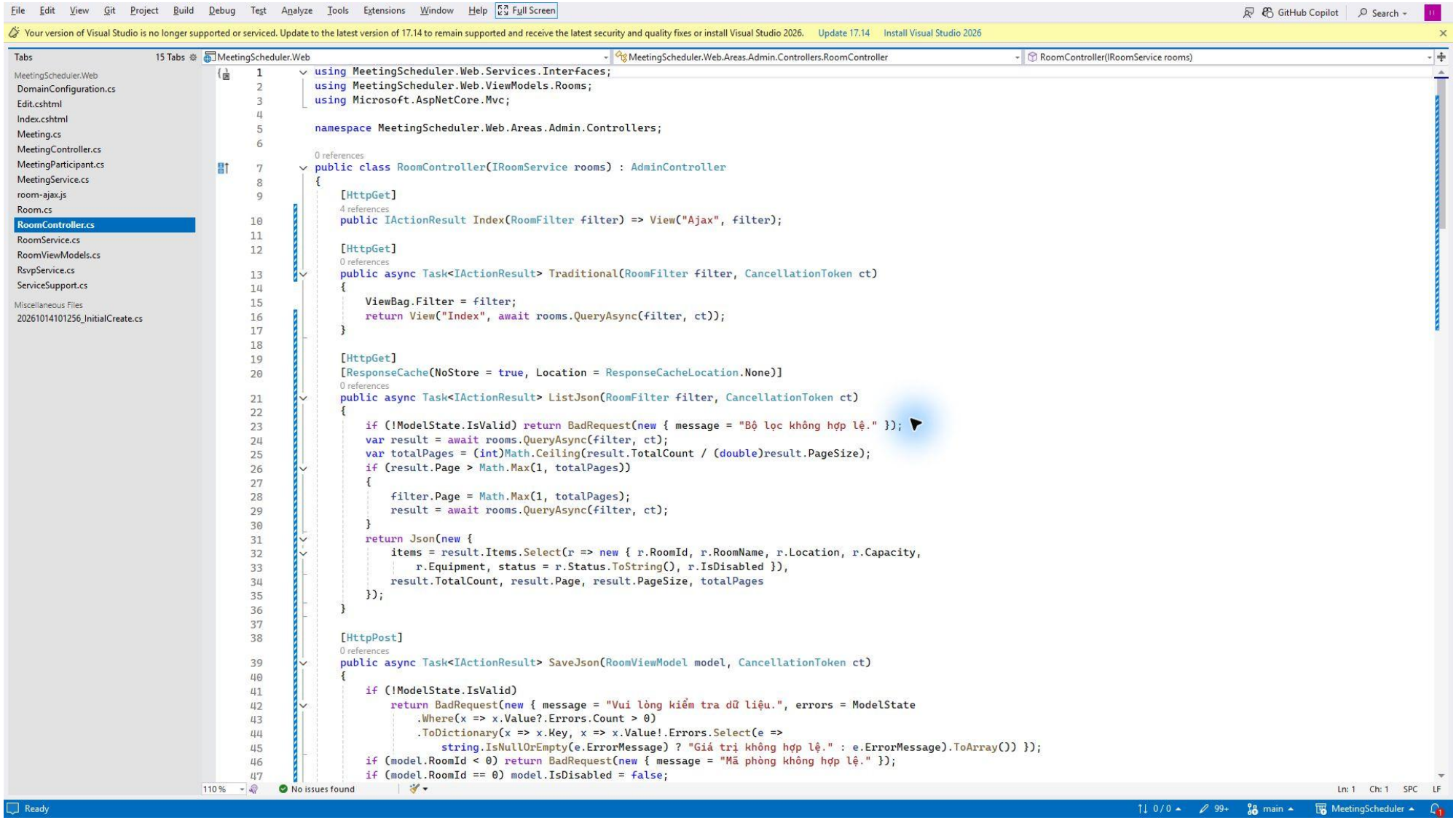
Ảnh code 4 Xóa mềm tìm kiếm và phân trang

File: wwwroot/js/room-ajax.js. Nút ngừng dùng/khôi phục gọi write() kèm id và token. Tìm kiếm debounce gọi load(fetch); nút trước/sau đổi page và gọi load() mặc định XMLHttpRequest.



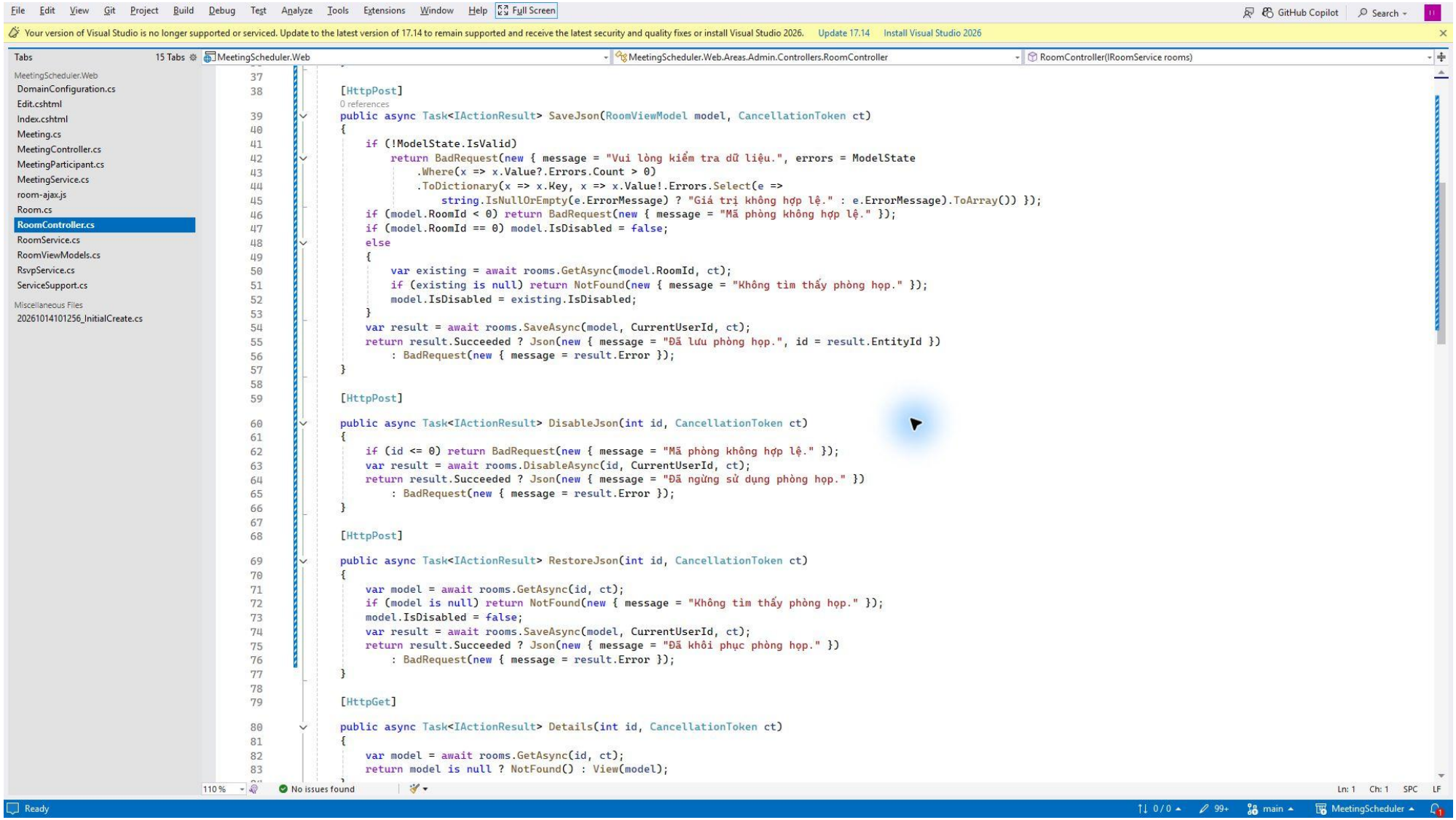
Ảnh code 5 Controller trả danh sách JSON

File: Areas/Admin/Controllers/RoomController.cs. ListJson nhận RoomFilter, gọi Service, điều chỉnh trang ngoài phạm vi và trả items, totalCount, page, pageSize, totalPages; không trả Razor HTML.



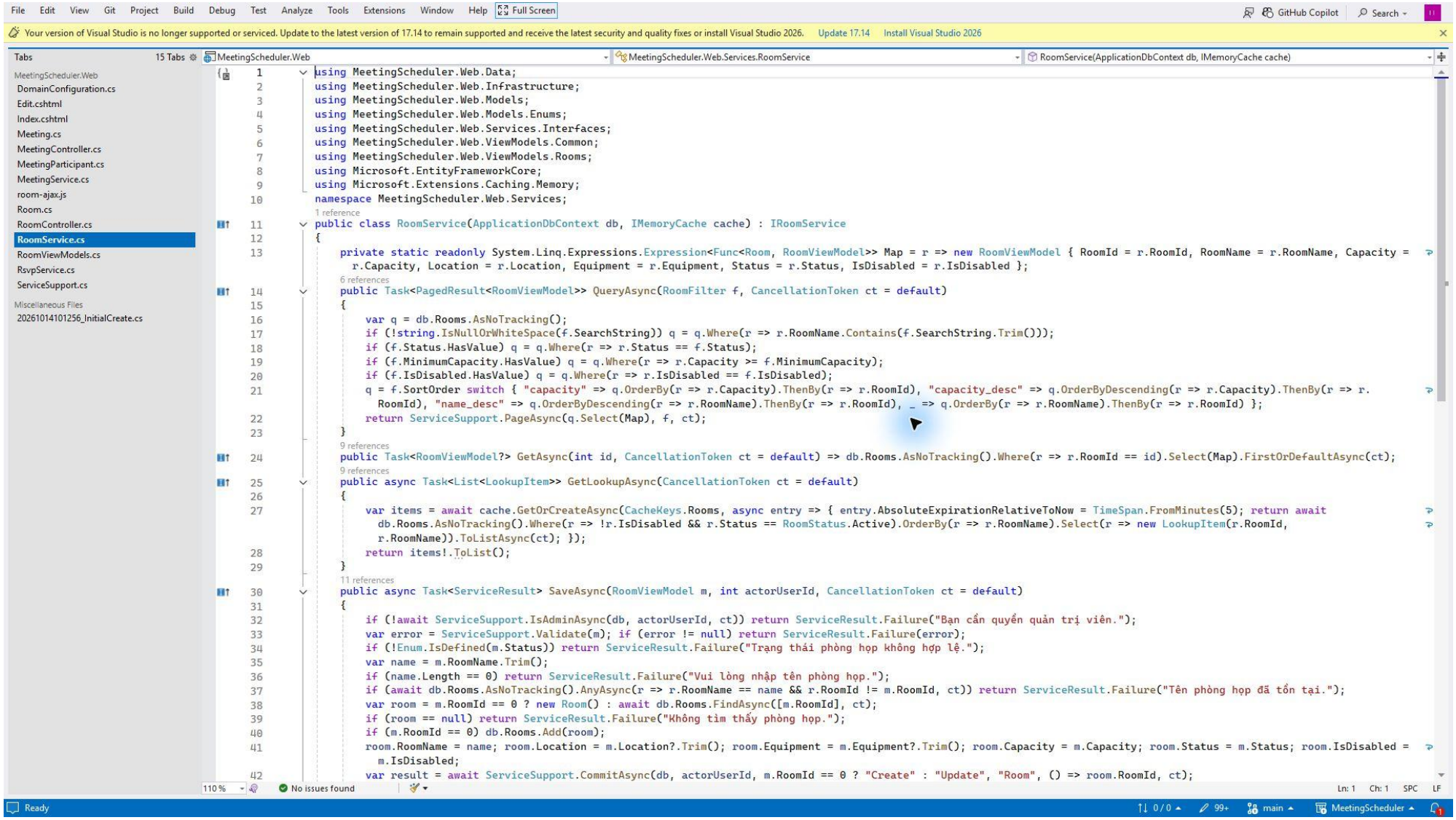
Ảnh code 6 Controller ghi dữ liệu và trả JSON

File: Areas/Admin/Controllers/RoomController.cs. SaveJson kiểm tra ModelState và mã phòng, gọi SaveAsync; DisableJson gọi DisableAsync; RestoreJson đặt IsDisabled=false. Lỗi trả HTTP 400 hoặc 404 với JSON.



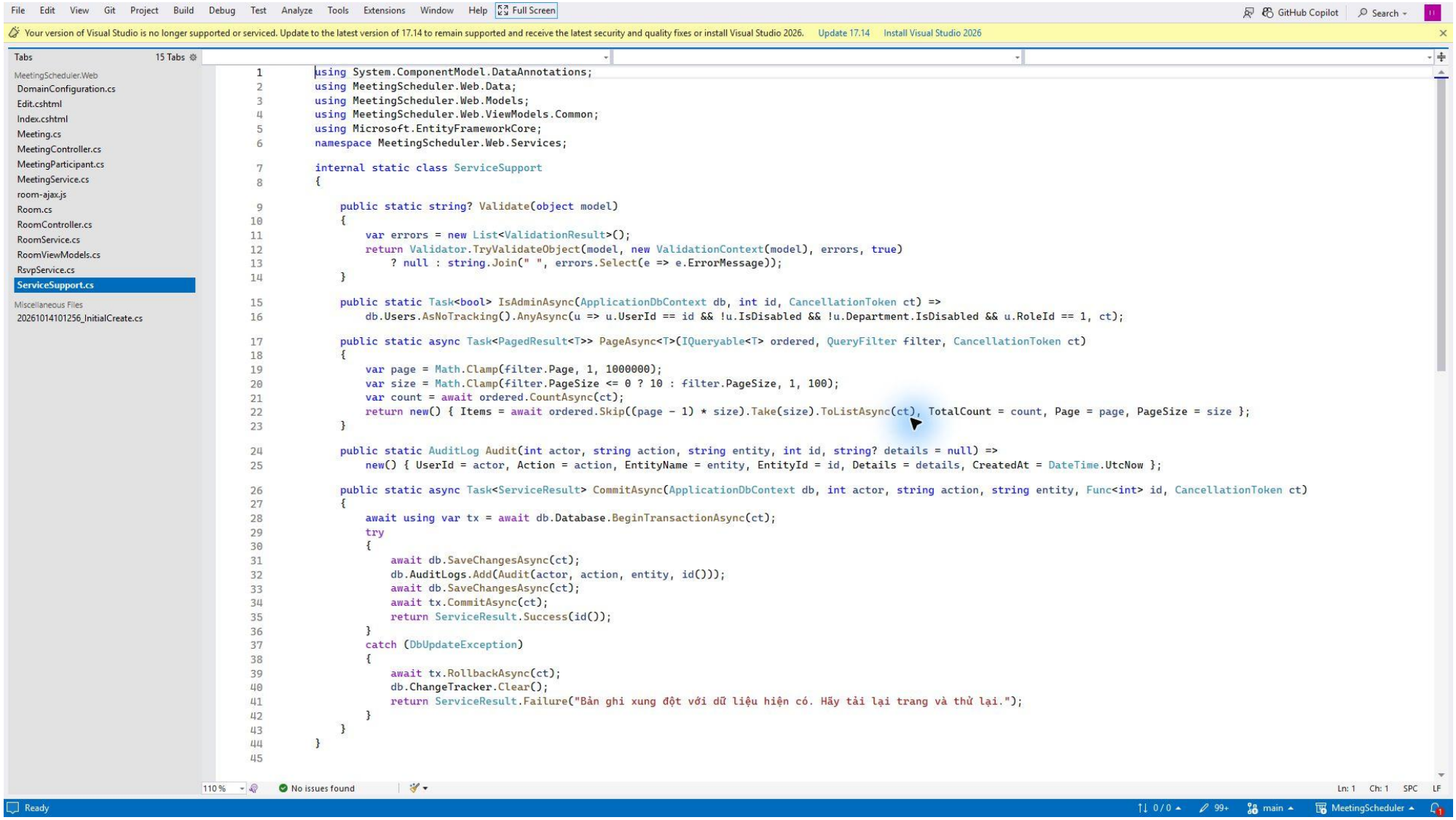
Ảnh code 7 LINQ lọc và sắp xếp

File: Services/RoomService.cs. QueryAsync dựng AsNoTracking, Where, OrderBy, Select rồi gọi ServiceSupport.PageAsync; dữ liệu vẫn được xử lý ở server qua EF Core.



Ảnh code 8 Phân trang bằng EF Core

File: Services/ServiceSupport.cs. PageAsync đếm CountAsync trước phân trang, sau đó Skip((page-1)*size), Take(size), ToListAsync. Đây là helper dùng chung được module Room gọi.



```
using System.ComponentModel.DataAnnotations;
using MeetingScheduler.Web.Data;
using MeetingScheduler.Web.Models;
using MeetingScheduler.Web.ViewModels.Common;
using Microsoft.EntityFrameworkCore;
namespace MeetingScheduler.Web.Services;

internal static class ServiceSupport
{

    public static string? Validate(object model)
    {
        var errors = new List<ValidationResult>();
        return Validator.TryValidateObject(model, new ValidationContext(model), errors, true)
            ? null : string.Join(" ", errors.Select(e => e.ErrorMessage));
    }

    public static Task<bool> IsAdminAsync(ApplicationDbContext db, int id, CancellationToken ct) =>
        db.Users.AsNoTracking().AnyAsync(u => u.UserId == id && !u.IsDisabled && !u.Department.IsDisabled && u.RoleId == 1, ct);

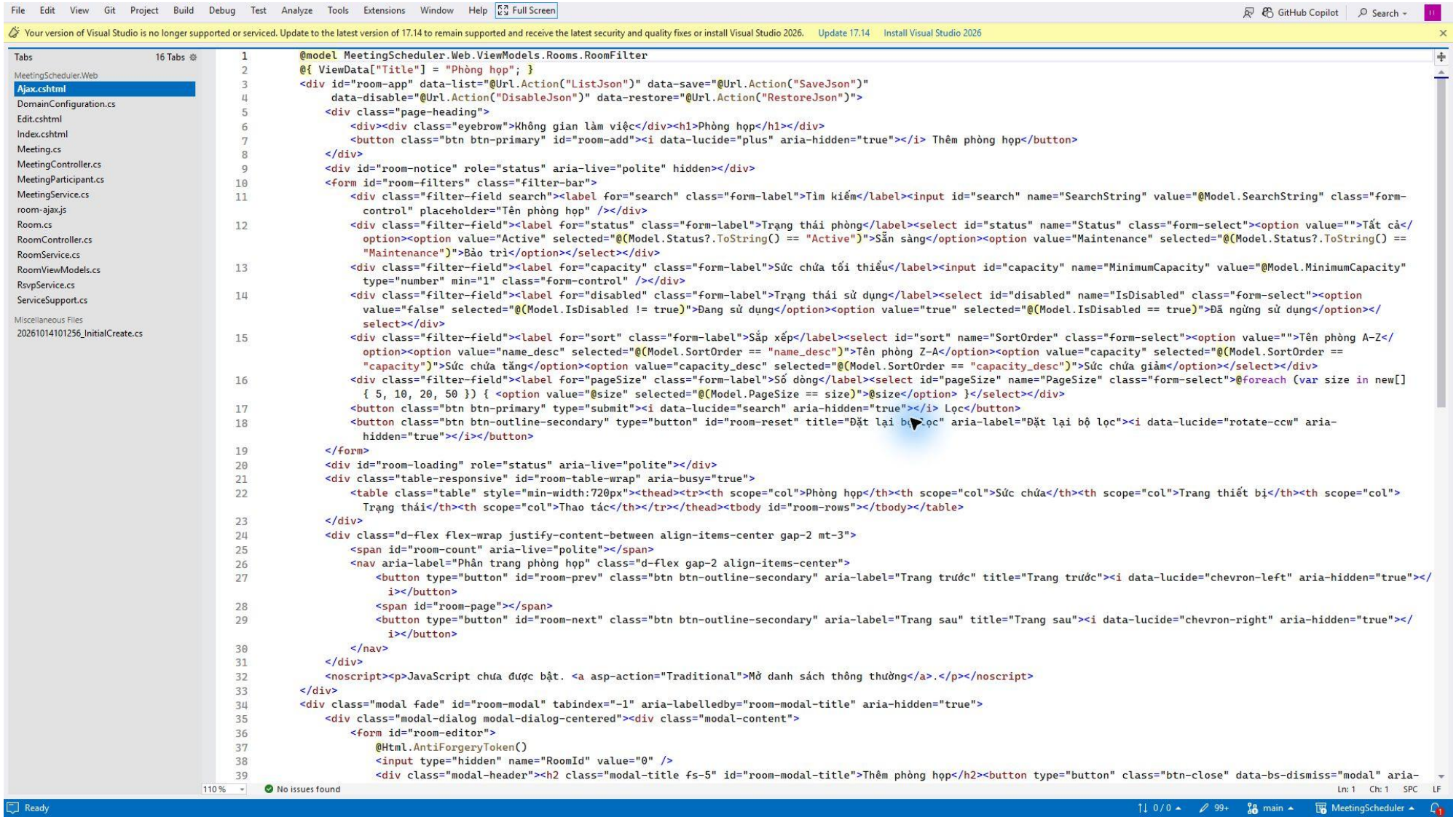
    public static async Task<PagedResult<T>> PageAsync<T>(IQueryable<T> ordered, QueryFilter filter, CancellationToken ct)
    {
        var page = Math.Clamp(filter.Page, 1, 1000000);
        var size = Math.Clamp(filter.PageSize <= 0 ? 10 : filter.PageSize, 1, 100);
        var count = await ordered.CountAsync(ct);
        return new() { Items = await ordered.Skip((page - 1) * size).Take(size).ToListAsync(ct), TotalCount = count, Page = page, PageSize = size };
    }

    public static AuditLog Audit(int actor, string action, string entity, int id, string? details = null) =>
        new() { UserId = actor, Action = action, EntityName = entity, EntityId = id, Details = details, CreatedAt = DateTime.UtcNow };

    public static async Task<ServiceResult> CommitAsync(ApplicationDbContext db, int actor, string action, string entity, Func<int> id, CancellationToken ct)
    {
        await using var tx = await db.Database.BeginTransactionAsync(ct);
        try
        {
            await db.SaveChangesAsync(ct);
            db.AuditLogs.Add(Audit(actor, action, entity, id()));
            await db.SaveChangesAsync(ct);
            await tx.CommitAsync(ct);
            return ServiceResult.Success(id());
        }
        catch (DbUpdateException)
        {
            await tx.RollbackAsync(ct);
            db.ChangeTracker.Clear();
            return ServiceResult.Failure("Bản ghi xung đột với dữ liệu hiện có. Hãy tải lại trang và thử lại.");
        }
    }
}
```

Ảnh code 9 Razor kết nối giao diện với JavaScript

File: Areas/Admin/Views/Room/Ajax.cshtml. data-list/save/disable/restore chứa URL endpoint; form bộ lọc, tbody room-rows, nút phân trang và antiforgery token được JavaScript sử dụng.



```
@model MeetingScheduler.Web.ViewModels.Rooms.RoomFilter
@{ ViewData["Title"] = "Phòng họp"; }
<div id="room-app" data-list="@Url.Action("ListJson")" data-save="@Url.Action("SaveJson")"
     data-disable="@Url.Action("DisableJson")" data-restore="@Url.Action("RestoreJson")">
    <div class="page-heading">
        <div><div class="eyebrow">Không gian làm việc</div><h1>Phòng họp</h1></div>
        <button class="btn btn-primary" id="room-add"><i data-lucide="plus" aria-hidden="true"></i> Thêm phòng họp</button>
    </div>
    <div id="room-notice" role="status" aria-live="polite" hidden></div>
    <form id="room-filters" class="filter-bar">
        <div class="filter-field search"><label for="search" class="form-label">Tìm kiếm</label><input id="search" name="SearchString" value="@Model.SearchString" class="form-
          control" placeholder="Tên phòng họp" /></div>
        <div class="filter-field"><label for="status" class="form-label">Trạng thái phòng</label><select id="status" name="Status" class="form-select"><option value="">Tất cả</
          option><option value="Active" selected="@(Model.Status?.ToString() == "Active")">Sẵn sàng</option><option value="Maintenance" selected="@(Model.Status?.ToString() ==
          "Maintenance")">Bảo trì</option></select></div>
        <div class="filter-field"><label for="capacity" class="form-label">Sức chứa tối thiểu</label><input id="capacity" name="MinimumCapacity" value="@Model.MinimumCapacity"
          type="number" min="1" class="form-control" /></div>
        <div class="filter-field"><label for="disabled" class="form-label">Trạng thái sử dụng</label><select id="disabled" name="IsDisabled" class="form-select"><option
          value="false" selected="@(Model.IsDisabled != true)">Đang sử dụng</option><option value="true" selected="@(Model.IsDisabled == true)">Đã ngừng sử dụng</option></
          select></div>
        <div class="filter-field"><label for="sort" class="form-label">Sắp xếp</label><select id="sort" name="SortOrder" class="form-select"><option value="">Tên phòng A–Z</
          option><option value="name_desc" selected="@(Model.SortOrder == "name_desc")">Tên phòng Z–A</option><option value="capacity" selected="@(Model.SortOrder ==
          "capacity")">Sức chứa tăng</option><option value="capacity_desc" selected="@(Model.SortOrder == "capacity_desc")">Sức chứa giảm</option></select></div>
        <div class="filter-field"><label for="pageSize" class="form-label">Số dòng</label><select id="pageSize" name="PageSize" class="form-select">@foreach (var size in new[]
          { 5, 10, 20, 50 }) { <option value="@size" selected="@(Model.PageSize == size)">@size</option> }</select></div>
        <button class="btn btn-primary" type="submit"><i data-lucide="search" aria-hidden="true"></i> Lọc</button>
        <button class="btn btn-outline-secondary" type="button" id="room-reset" title="Đặt lại bộ lọc" aria-label="Đặt lại bộ lọc"><i data-lucide="rotate-ccw" aria-
          hidden="true"></i></button>
    </form>
    <div id="room-loading" role="status" aria-live="polite"></div>
    <div class="table-responsive" id="room-table-wrap" aria-busy="true">
        <table class="table" style="min-width:720px"><thead><tr><th scope="col">Phòng họp</th><th scope="col">Sức chứa</th><th scope="col">Trang thiết bị</th><th scope="col">
          Trạng thái</th><th scope="col">Thao tác</th></tr></thead><tbody id="room-rows"></tbody></table>
    </div>
    <div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mt-3">
        <span id="room-count" aria-live="polite"></span>
        <nav aria-label="Phân trang phòng họp" class="d-flex gap-2 align-items-center">
            <button type="button" id="room-prev" class="btn btn-outline-secondary" aria-label="Trang trước" title="Trang trước"><i data-lucide="chevron-left" aria-hidden="true"></
              i></button>
            <span id="room-page"></span>
            <button type="button" id="room-next" class="btn btn-outline-secondary" aria-label="Trang sau" title="Trang sau"><i data-lucide="chevron-right" aria-hidden="true"></
              i></button>
        </nav>
    </div>
    <noscript><p>JavaScript chưa được bật. <a asp-action="Traditional">Mở danh sách thông thường</a>.</p></noscript>
</div>
<div class="modal fade" id="room-modal" tabindex="-1" aria-labelledby="room-modal-title" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered"><div class="modal-content">
        <form id="room-editor">
            @Html.AntiForgeryToken()
            <input type="hidden" name="RoomId" value="0" />
            <div class="modal-header"><h2 class="modal-title fs-5" id="room-modal-title">Thêm phòng họp</h2><button type="button" class="btn-close" data-bs-dismiss="modal" aria-
```

3 Data Flow từng thao tác

Lần tải đầu: GET Index trả khung Razor, form và token. JavaScript gọi XMLHttpRequest GET ListJson; controller nhận RoomFilter, gọi RoomService.QueryAsync. Service dựng LINQ AsNoTracking, Where, OrderBy và phân trang. EF Core gửi SQL tới database, kết quả được chiếu sang RoomViewModel. Controller trả JSON; JavaScript dựng bảng và cập nhật số trang. Chỉ lần tải đầu có HTML document mới.

Tìm kiếm/lọc: UI nhập tên hoặc chọn trạng thái/sức chứa -> JS xây URLSearchParams -> fetch GET ListJson -> ModelState -> Service -> LINQ/SQL -> JSON -> response.json -> render tbody. AbortController và sequence loại bỏ kết quả request cũ khi tìm kiếm liên tục. Không gửi form GET làm điều hướng trang.

Thêm: bấm Thêm phòng -> modal RoomViewModel -> submit event preventDefault -> kiểm tra HTML5 -> $.ajax POST SaveJson + token -> kiểm tra ModelState và quyền Admin -> RoomService.SaveAsync kiểm tra tên trùng, enum, sức chứa -> INSERT Room và AuditLog cùng transaction -> commit và xóa cache -> JSON thành công -> đóng modal và tải lại bảng. Lỗi nghiệp vụ trả HTTP 400 JSON và hiện trong modal, dữ liệu nhập được giữ lại.

Sửa: bấm biểu tượng bút -> nạp dữ liệu hàng vào modal -> POST SaveJson với RoomId -> kiểm tra bản ghi tồn tại -> giữ IsDisabled hiện tại -> SaveAsync UPDATE các trường được phép và AuditLog -> JSON -> cập nhật bảng. Đây là cập nhật lạc quan theo hành vi Room hiện có; không bổ sung RowVersion cho Room.

Ngừng dùng: bấm biểu tượng ngừng -> xác nhận -> $.ajax POST DisableJson(id, token) -> DisableAsync đặt IsDisabled = true -> UPDATE + AuditLog -> commit/xóa cache -> JSON -> tải lại danh sách. Không DELETE Room. Nếu xóa hàng cuối trang, ListJson tự đưa về trang còn dữ liệu. Chọn Đã ngừng sử dụng để tìm và khôi phục qua RestoreJson.

4 JSON và HTTP

GET ListJson trả items gồm roomId, roomName, location, capacity, equipment, status, isDisabled; kèm totalCount, page, pageSize, totalPages. Status xuất rõ chuỗi Active hoặc Maintenance.

POST SaveJson thành công trả message và id. Dữ liệu sai trả HTTP 400 với message và errors theo tên trường; lỗi tên trùng trả message. Sửa mã không tồn tại trả 404. DisableJson/RestoreJson thành công trả message. Các endpoint thừa kế quyền Admin và bộ lọc antiforgery của ứng dụng. Tài khoản không có quyền nhận 403; phiên hết hạn được phát hiện khi JSON không hợp lệ/không đúng content-type, giao diện yêu cầu kiểm tra đăng nhập.

Không lấy actorUserId từ form; CurrentUserId lấy từ phiên đăng nhập. Dữ liệu người dùng render bằng textContent để không thực thi HTML. Các nút ghi bị khóa trong khi đang gửi để hạn chế bấm lặp. Có trạng thái đang tải, rỗng, lỗi kết nối và thông báo thành công; giữ thứ tự request để tránh ghi đè kết quả tìm kiếm mới bằng phản hồi cũ.

5 AJAX và request truyền thống

Request truyền thống gửi form, server trả View hoặc redirect và trình duyệt tải lại HTML. Với AJAX, JavaScript gửi request nền; server trả JSON, JavaScript chỉ cập nhật phần DOM cần đổi. Database và nghiệp vụ vẫn nằm phía server, không chuyển SQL hay quyền ghi xuống JavaScript. XMLHttpRequest là API gốc dùng callback; fetch là API gốc dùng Promise/async-await; jQuery

$.ajax cung cấp cấu hình gửi form và xử lý lỗi thuận tiện. Cả ba đều gọi chung Controller/Service, không phải ba cơ chế truy cập database khác nhau.

6 Chạy và kiểm tra

Mở MeetingScheduler.sln bằng Visual Studio 2022, đặt MeetingScheduler.Web làm Startup Project, profile http, F5. Hoặc chạy dotnet run --project MeetingScheduler.Web từ thư mục solution. Đăng nhập admin@company.test / Demo@12345 và mở /Admin/Room. Các tài khoản mẫu chỉ dùng trong môi trường học tập.

Ngày 21/10/2026, Codex build web thành công, 0 warning/0 error; 9 nhóm kiểm tra tích hợp backend đạt. Kiểm tra trình duyệt chạy trên cùng source với database kiểm tra riêng, không sửa dữ liệu database làm việc: có thêm/sửa, trùng tên, ngừng dùng/khôi phục, tìm kiếm, trang 2, chống XSS, chống POST thiếu token và kiểm tra quyền. Bộ đếm request xác nhận các thao tác AJAX không điều hướng/reload document. Các ảnh trong Minh_chung là ảnh chạy thật bằng Edge/Playwright, không phải ảnh thiết kế; sinh viên cần tự chạy lại trước khi trình bày.

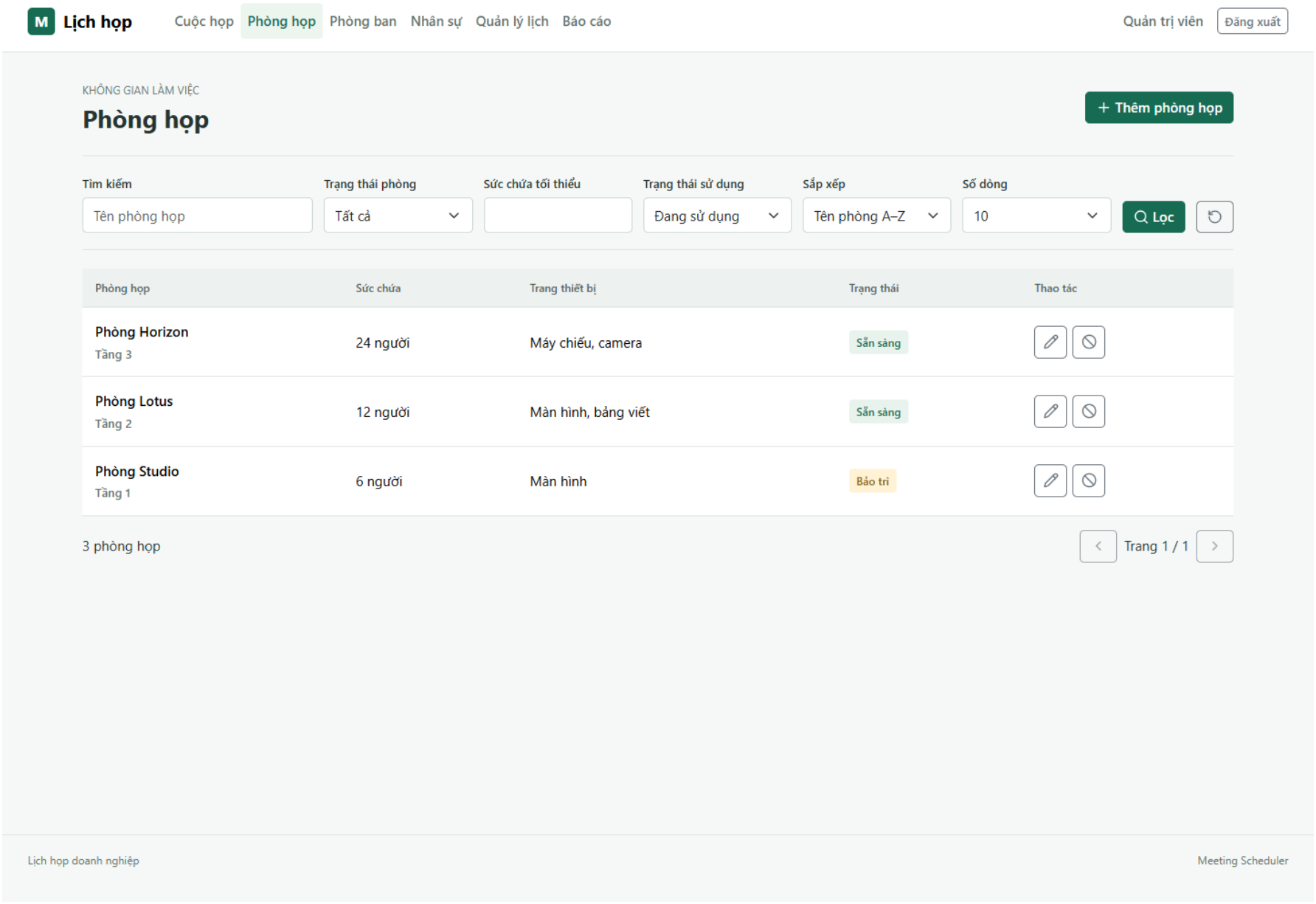
Chi tiết kiểm tra: scripts/check_room_ajax.cjs và Minh_chung/ket_qua_kiem_tra.json. Ảnh chỉ minh họa dữ liệu demo của phiên kiểm tra, không khẳng định sinh viên đã tự thao tác.

7 File nộp và phần dùng chung

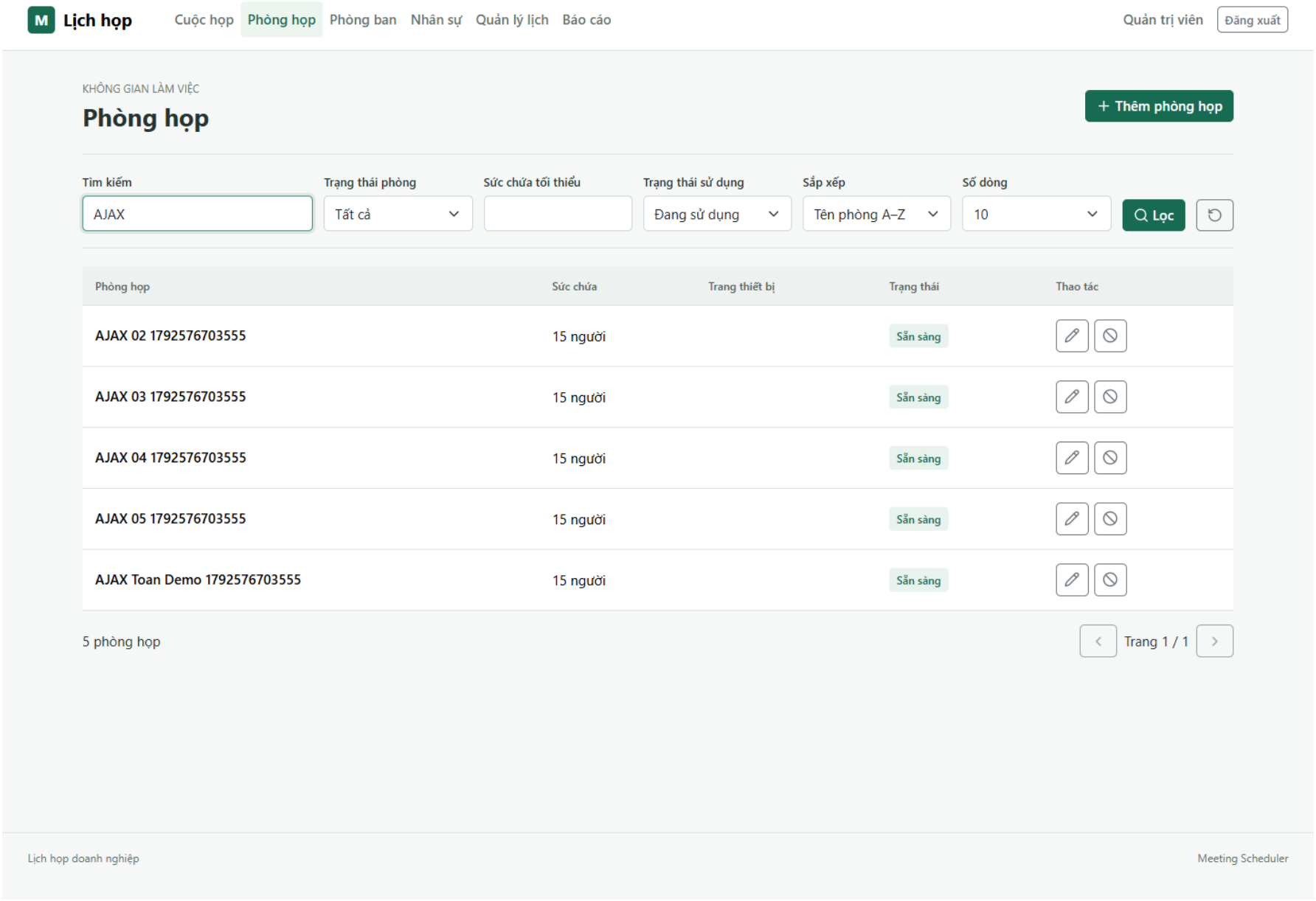
Mã bổ sung: Areas/Admin/Controllers/RoomController.cs; Areas/Admin/Views/Room/Ajax.cshtml; wwwroot/js/room-ajax.js. Backend dùng lại Services/RoomService.cs, Services/ServiceSupport.cs, ViewModels/Rooms/RoomViewModels.cs, ApplicationDbContext và Model Room.

Folder cá nhân Ajax_Room gồm báo cáo Word/Markdown, ảnh giao diện, kết quả kiểm tra và ZIP source đầy đủ để chạy. ZIP bao gồm khung chung nhưng không xem module của thành viên khác là phần cá nhân của Toàn. Không đóng gói bin/obj/.vs/Logs hay database.

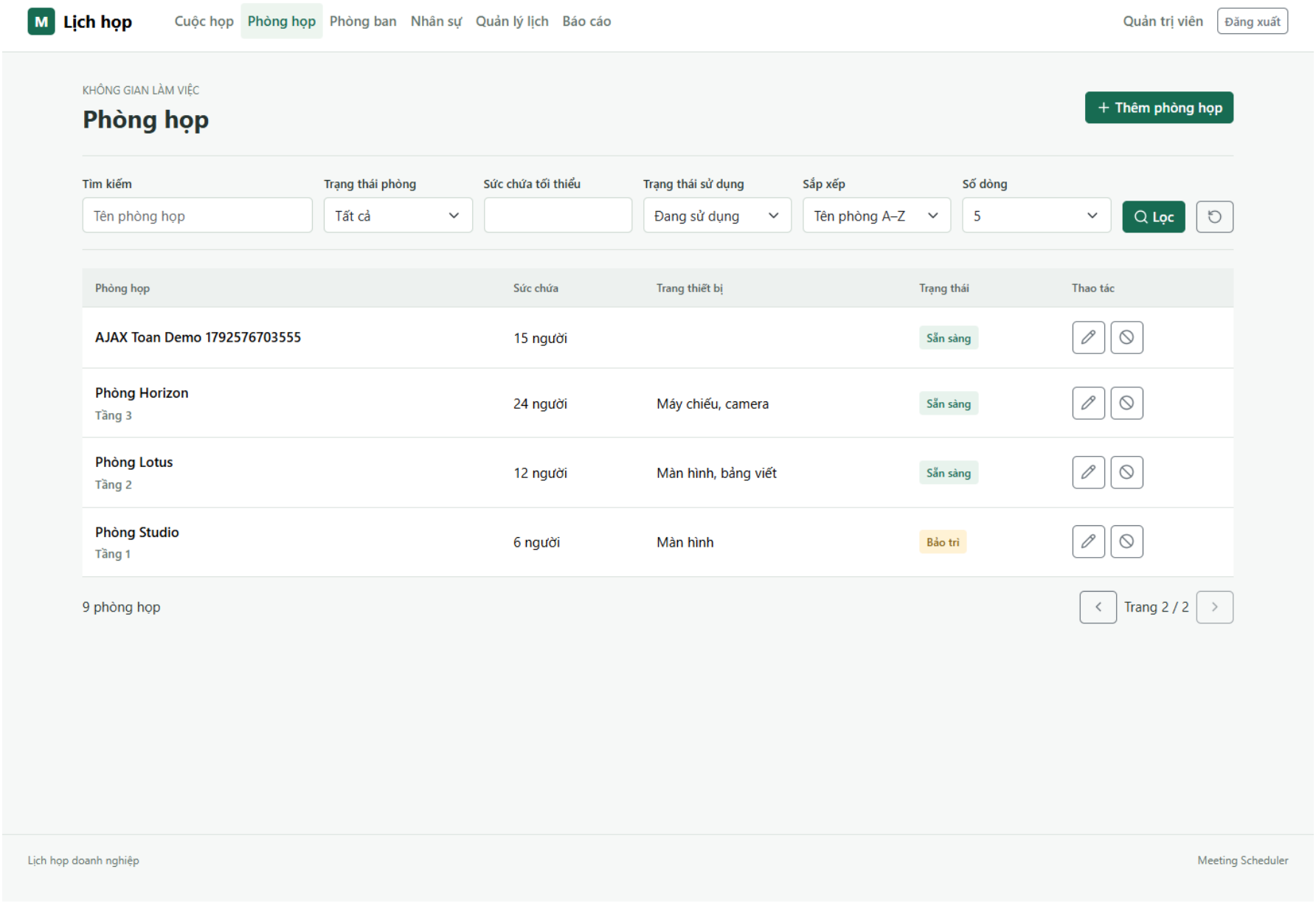
Minh chứng 1 Danh sách phòng tải bằng XMLHttpRequest



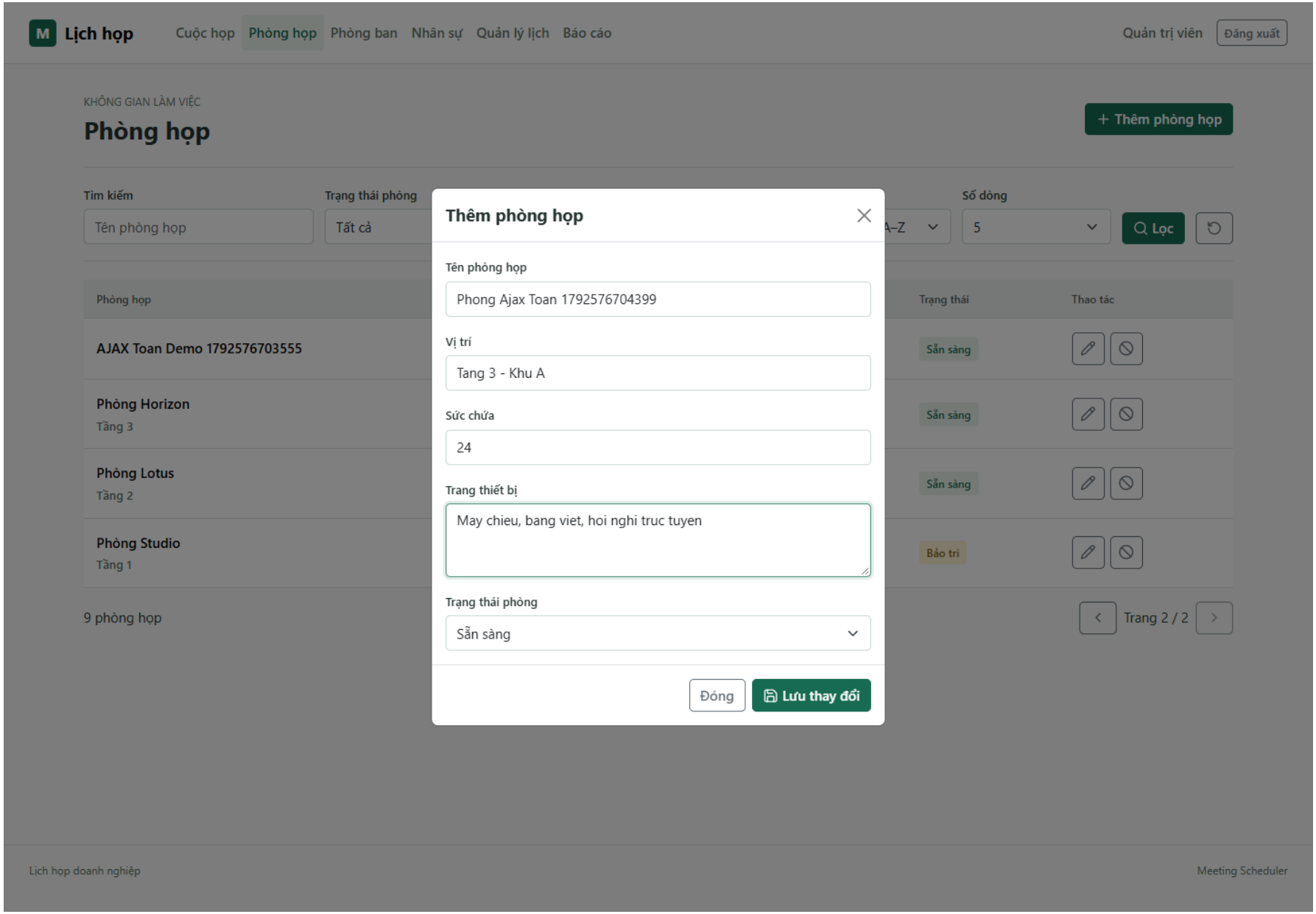
Minh chứng 2 Tìm kiếm và cập nhật bảng bằng fetch



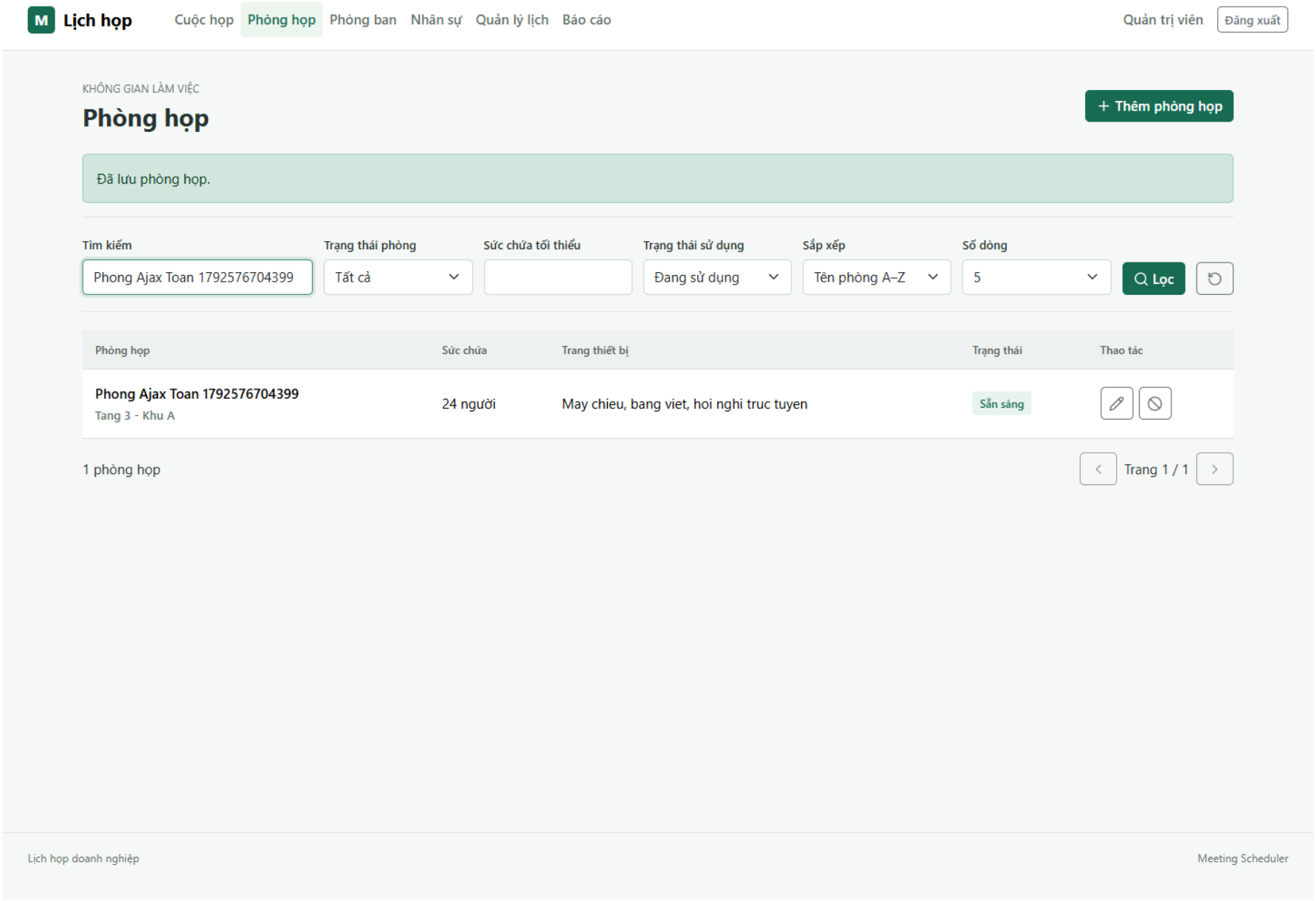
Minh chứng 3 Tải trang thứ hai không reload trang



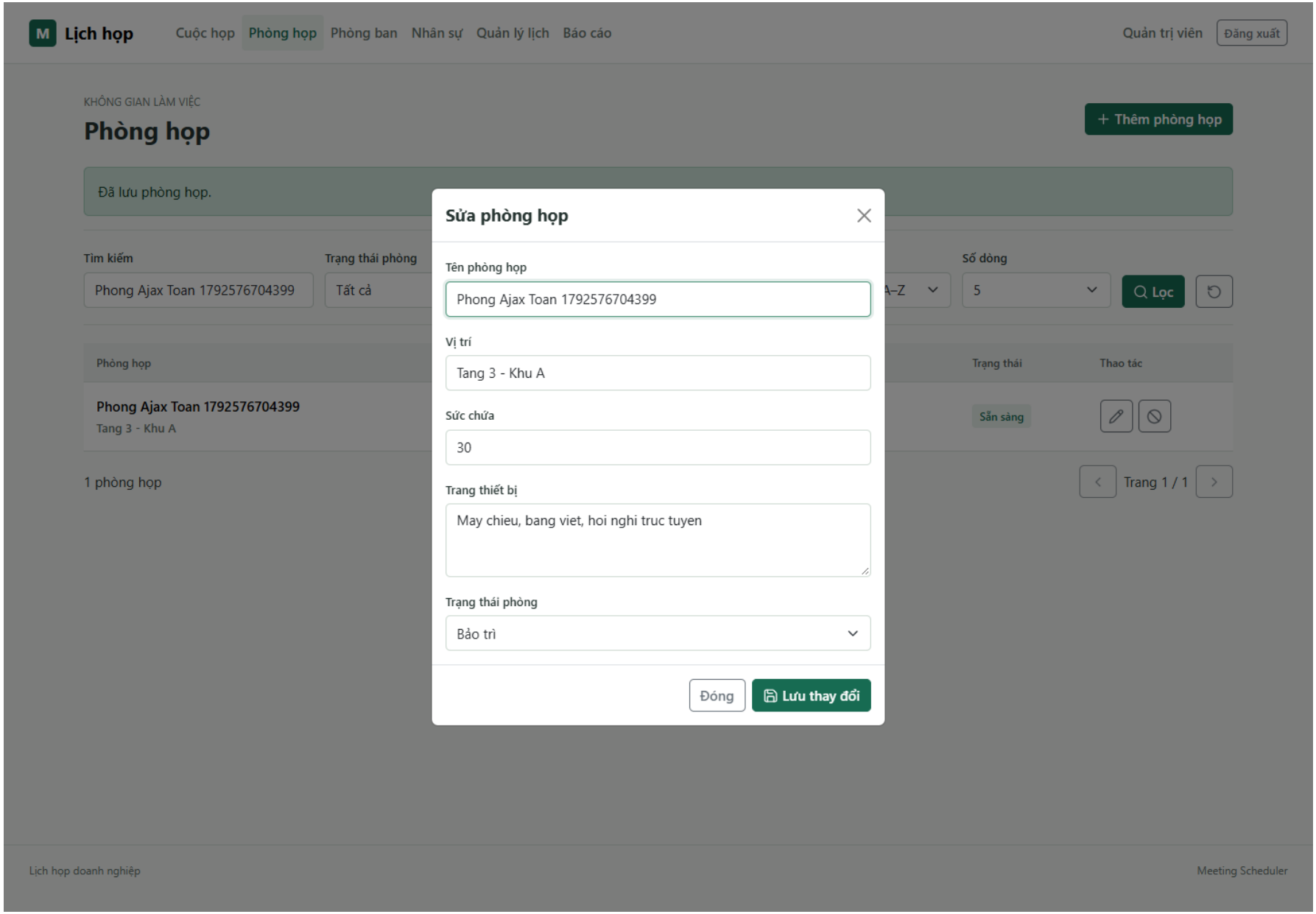
Minh chứng 4 Biểu mẫu thêm phòng



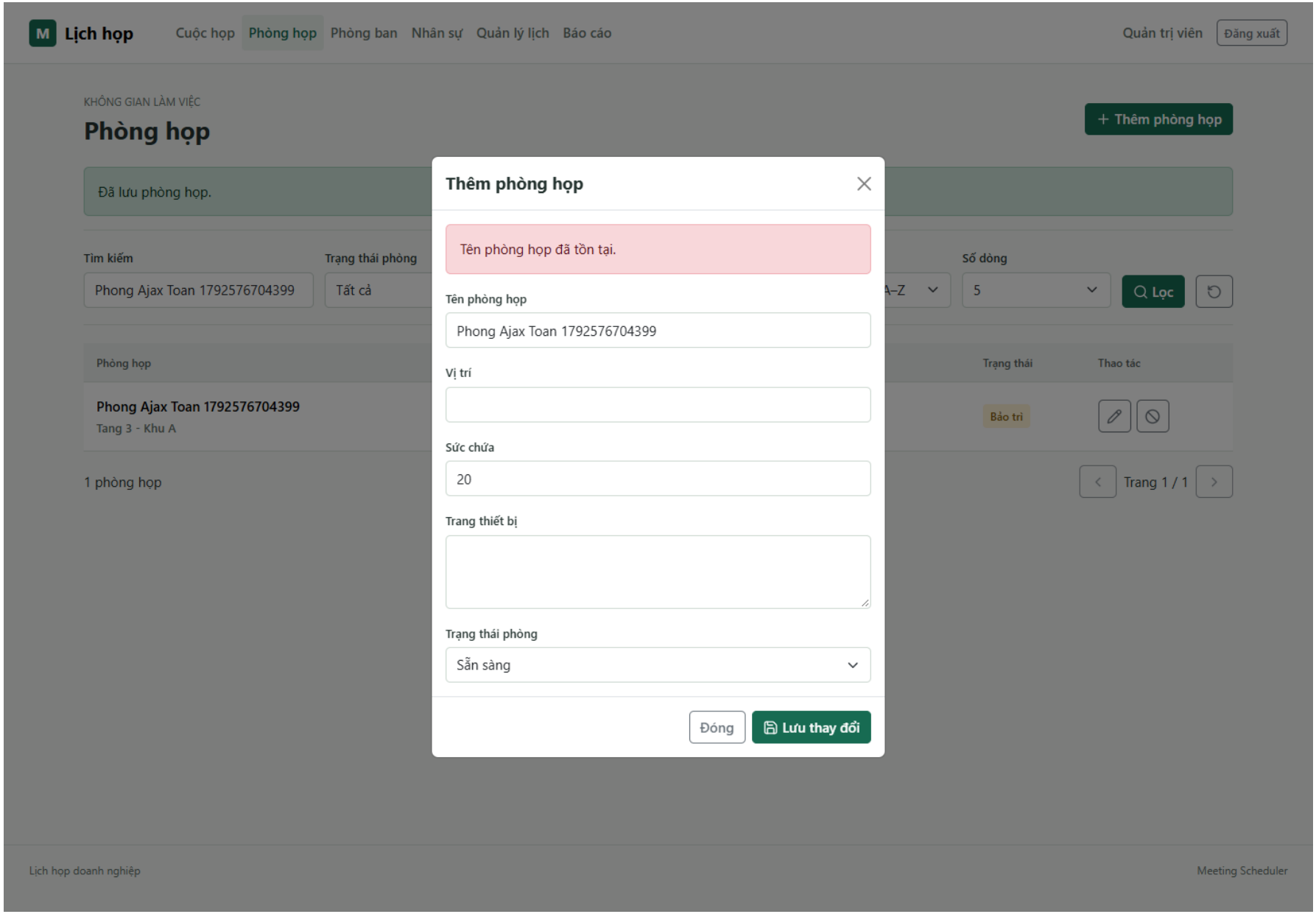
Minh chứng 5 Kết quả lưu thành công qua jQuery AJAX



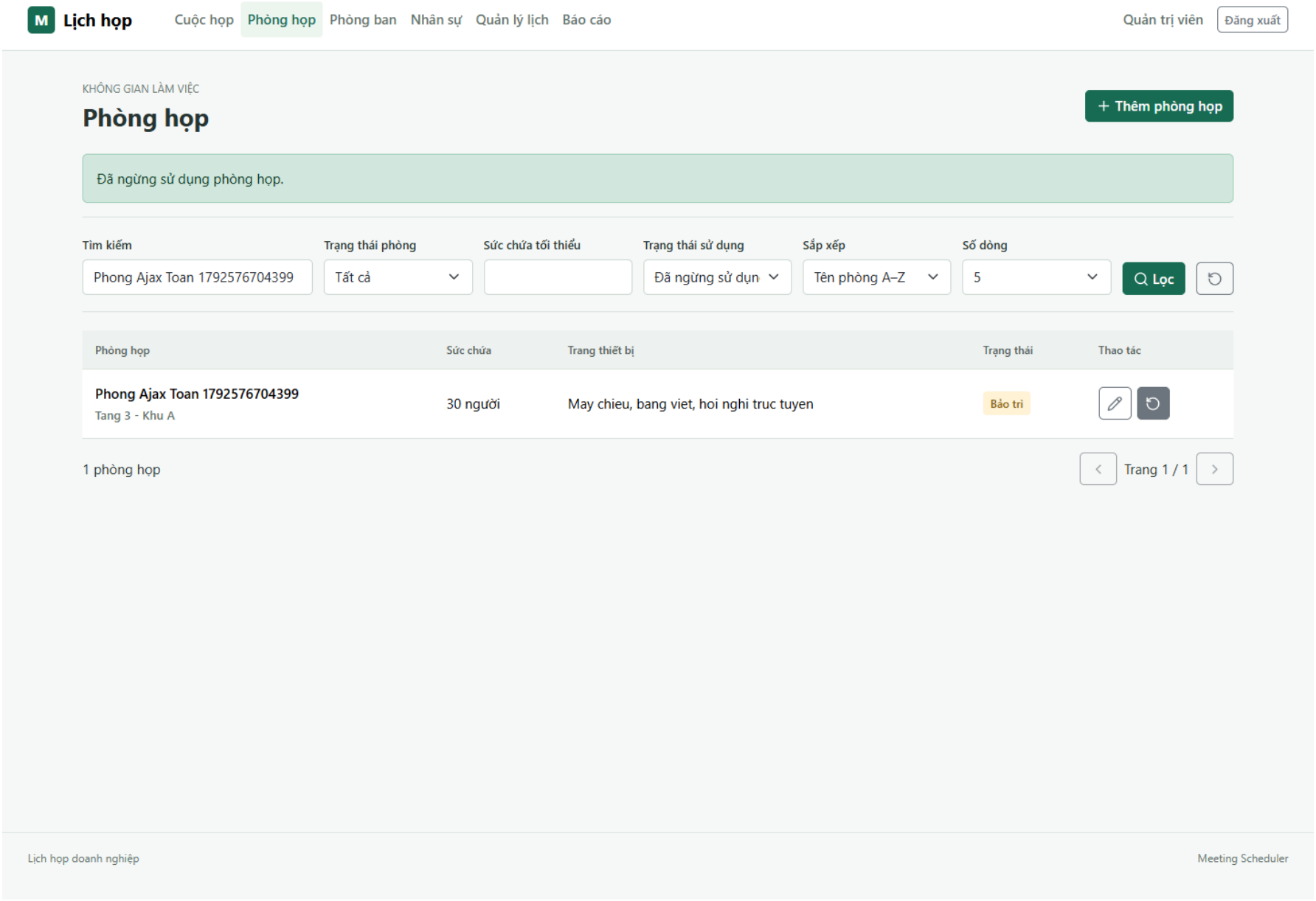
Minh chứng 6 Sửa sức chứa và chuyển sang bảo trì



Minh chứng 7 Lỗi tên trùng hiển thị trong modal



Minh chứng 8 Phòng ngừng dùng vẫn được lưu trong database



Minh chứng 9 Khôi phục phòng và tải lại bảng

